ELE8088: Control & Estimation Theory **QUB, 2023/24**

Assignment A5: Model Predictive Control Design

Important: Before you start, read the instructions on Canvas!

1 Computational tools

In this assignment you need to use the following tools.

1.1 Solving quadratic optimisation problems in Python

The Python module `qpsolvers`[1] can be used to solve convex quadratic optimisation problems of the standard form

$$\operatorname*{minimise}_{x\in\mathbb{R}^n} \tfrac{1}{2}x^\intercal Px + q^\intercal x, \tag{1a}$$

$$\text{subject to: } Gx \leq h, \tag{1b}$$

$$Ax = b, \tag{1c}$$

$$x_{\min} \leq x \leq x_{\max}, \tag{1d}$$

where $P \in \mathbb{S}^n_{++}$, $q \in \mathbb{R}^n$, $G \in \mathbb{R}^{p\times n}$, $h \in \mathbb{R}^p$, $A \in \mathbb{R}^{s\times n}$, $b \in \mathbb{R}^s$, and $x_{\min}, x_{\max} \in \mathbb{R}^n$. In (1b) and (1d), $\leq$ is meant in the element-wise sense. Read the documentation for details.

1.2 Operations with polyhedra

A polyhedron is a nonempty convex set of the form $E = \{x \in \mathbb{R}^n : Fx \leq f\}$, for $F \in \mathbb{R}^{m\times n}$ and $f \in \mathbb{R}^m$. If a polyhedron is compact, we typically call it a *polytope*. In Python, the module `polyope`[2] can be used to perform certain basic operations with polyhedra and polytopes. For example, using the function `polytope.extreme` we can determine the extreme points — or *vertices* — of a polytope. The vertices of a polytope E are points $v_i \in E$, $i = 1, \ldots, n_v$, which cannot be written as a convex combination of other points in E[3]. We can also use `polyope` to check whether a polytope is a subset of another, to plot polytopes (this works only in two dimensions), to eliminate redundant constraints (see Section 10.3.4) to perform projection operations (see Secton 10.4), and many more. You should read the documentation for details.

[1] For documentation and installation instructions, see `https://github.com/qpsolvers/qpsolvers`.

[2] See `https://github.com/tulip-control/polytope` for documentation. To install it using pip, run `pip install polytope`.

[3] As an example, consider the unit $\|\cdot\|_\infty$-ball in $\mathbb{R}^2$, which is the set $B = \{x \in \mathbb{R}^2 : \|x\|_\infty \leq 1\}$. This can be written as $B = \{x \in \mathbb{R}^2 : Fx \leq f\}$, for $F = \left[\begin{smallmatrix} I_2 \\ -I_2 \end{smallmatrix}\right]$ and $f = 1_4$. The extreme points of B are the points $(-1,-1)$, $(-1,1)$, $(1,-1)$, and $(1,1)$.

2 Problem statement

You need to design a control system for the pitch of an aircraft. The system is illustrated in Figure 1. The state variables are the angle of attack, α, the pitch rate, q, and the pitch angle, θ. The manipulated input is the elevator deflection angle, δ.

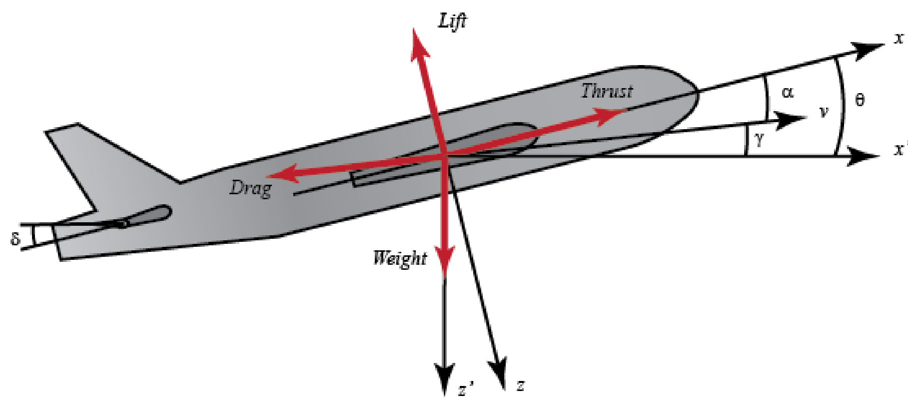

Figure 1: Illustration of an aircraft with its pitch angle, θ, the angle of attack, α, and the deflection angle of the elevators, δ. The angle $\gamma = \theta - \alpha$ is known as the *slope*.

The system dynamics is given by[4]

$$\begin{bmatrix}\alpha_{t+1}\\ q_{t+1}\\ \theta_{t+1}\end{bmatrix} = \begin{bmatrix}0.9835000 & 2.782 & 0\\ -0.0006821 & 0.978 & 0\\ -0.0009730 & 2.804 & 1\end{bmatrix}\begin{bmatrix}\alpha_t\\ q_t\\ \theta_t\end{bmatrix} + \begin{bmatrix}0.01293\\ 0.00100\\ 0.001425\end{bmatrix}\delta_t, \tag{2}$$

where all quantities are in SI units[5]. This dynamics has been obtained with a sampling time of 0.05 s. We assume that all three state variables can be measured without error[6].

The task of your team is to design a model predictive controller (MPC) for this system using the methodology given in Sections 10.2 – 10.5 of Handout 10. Your MPC should drive the state to the origin while guaranteeing the satisfaction of the following constraints

1. The angle of attack, in absolute value, must never exceed 11.5°
2. The deflection angle of the elevators, can only be between −24° and 27°
3. The pitch angle of the aircraft, in absolute value, must not exceed 35°
4. The pitch angle rate, in absolute value, must not exceed 14 deg/s
5. The slope, in absolute value, must not exceed 23°

[4]This is adapted from the *Control Tutorials for Matlab and Simulink* by Prof. Dawn Tilbury at the University of Michigan and Prof. Bill Messner at Carnegie Mellon.

[5]In Equation (2), α, θ and, δ are measured in radians — not degrees — and q is measured in radians per second.

[6]This is an unrealistic assumption. In practice, more often than not, we can only measure the pitch angle. Needless to say, our measurements are corrupted by noise. We address this case in the *Estimation* part of the module.

In addition, abrupt *changes* of the control action are undesirable. You need to take this into account when designing your MPC[7].

Use `qpsolvers` to solve the optimisation problem of MPC; to that end, you first need to write the problem in the standard form of Equation (1). Use the module `polytope` to compute the maximal invariant set.

In your technical report you should

1. Clearly state your MPC formulation (state the MPC optimisation problem) and explain how MPC works.
2. Justify all your design choices, including your choice of tuning parameters (e.g., cost function parameters, terminal ingredients, prediction horizon).
3. Explain how exactly you brought your MPC problem in the standard form of Equation (1). For example, you may or may not have decided to eliminate the sequence of states. In any case, you need to justify your choices.
4. Perform simulations to demonstrate that the closed-loop system works properly starting from all extreme points of the set X_N.
5. Discuss how fast you can solve the optimisation problem[8].
6. Discuss in detail the theoretical properties of the MPC-controlled with the proposed MPC controller.
7. Provide adequate technical details so that other people cam understand and replicate your results.

In addition, your code needs to be written clearly, be properly organised, and well documented.

Your report will be marked for technical correctness and completeness of the proposed approach, justification, clarity, quality of presentation; this will account for 90% of the marks of your report. Your report must include an appendix or section entitled "project management and individual contributions" where you should provide evidence on (i) how you collaborated (what collaboration tools did you use? For example, did you use git/GitHub?) (ii) how you managed the work in this project? How were the tasks distributed? Did you use an issue tracker? (iii) any additional information project management you would like to include (e.g., planning), (iv) what were the contributions of each member of the team?

Marks will only be awarded based on the **evidence** you will provide. Examples of appropriate evidence can be meeting minutes, or a link to a GitHub repository, or other similar online collaborative development space. This will account for 10% of the marks of your report.

[7] Consider using the methodology of Section 10.7 in Handout 10.

[8] Report your average and maximum computation times. Note that Google Colab can be slow. You may get better performance if you run the code locally on your computer.